ğrencibilgiLab 3.5 – Hafta 3.5OBYS Tam Proje – Derlenebilir CMake Projesi

Lab Hakkında

Bu laboratuvar çalışmasında Bölüm 3 Ünite 5'teki OBYS projesinin temel bileşenlerini adım adım oluşturacaksınız: CMake yapılandırması, struct varlıklar, template Depo sınıfı, servis katmanı, binary dosyalama ve birim testleri.

Süre: 100 dakika **Derleme:** `mkdir build && cd build && cmake .. && make`

1 Çalışma 1: CMake Proje Yapısı ve Struct'lar (20 dk)

Ön Bilgi – CMake ve Dizin Yapısı

CMake, C++ projelerini platformdan bağımsız olarak derlemenizi sağlayan bir yapı sistemidir. `CMakeLists.txt` dosyasında proje adı, C++ standardı ve kaynak dosyalar belirtilir. CMake bu bilgilerden `Makefile` üretir.

Dizin yapısı: Başlık dosyaları `include/` altında, kaynak dosyaları `src/` altında tutulur. Bu ayrım, başlıkların birden fazla hedef (`Uygulama3` ve `TestUygulama3`) tarafından paylaşılmasını sağlar.

Aşağıdaki dizin yapısını oluşturun:

```
obys_modern/
  CMakeLists.txt
  include/
    varliklar/
        renci   .h
      ders.h
      ders_kaydi.h
      not_turleri.h
  src/
    main.cpp
```

Başlangıç `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20)
project(Uygulama3 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

include_directories(include)

# GOREV: Hedefleri ekleyin
```

Başlangıç `include/varliklar/öğrenci.h`:

```
#pragma once
#include <string>

struct    renci    {
    int numara{};
    std::string isim;
    std::string soyisim;
    std::string dogum_tarihi;
    std::string cinsiyet;
};
```

Başlangıç `include/varliklar/ders.h`:

```cpp
#pragma once
#include <string>

struct Ders {
    std::string ders_kodu;
    std::string ders_adi;
};
```

Başlangıç `include/varliklar/not_turleri.h`:

```cpp
#pragma once
#include <variant>
#include <string>

struct Muaf {
    std::string sebep{"Muaf"};
};

using NotTuru = std::variant<int, std::string, Muaf>;
```

Görev 1.1 – CMakeLists.txt Tamamla (5 dk)

`CMakeLists.txt` dosyasına aşağıdaki hedefi ekleyin:

a) `add_executable(Uygulama3 src/main.cpp)` satırını ekleyin.

b) `main.cpp` dosyasında basit bir `Hello World` ile derleyin.

```
$ mkdir build && cd build
$ cmake ..
$ make
$ ./Uygulama3
```

Görev 1.2 – DersKaydi Struct'ı Yaz (10 dk)

`include/varliklar/ders_kaydi.h` dosyasını oluşturun:

a) `#pragma once` ve gerekli başlıkları ekleyin (`<string>`, `<variant>`, `"varliklar/not_turleri.h"`).

b) `DersKaydi` struct'ını tanımlayın:

- `int öğrenci_no{}` – öğrenci numarası
- `std::string ders_kodu` – ders kodu
- `NotTuru vize, final_notu, odev` – variant not türleri

c) `float ortalama() const` metodu ekleyin: vize %30 + final %40 + ödev %30 formülüyle hesaplasın.

d) `static int not_degeri(const NotTuru& n)` yardımcı fonksiyonunu ekleyin: `int` ise doğrudan döndür, `string` ise `stoi` ile dönüştür, `Muaf` ise 0 döndür.

variant Erişimi

`std::visit` ile variant'ın aktif türüne göre işlem yapabilirsiniz:

```
return std::visit([](const auto& val) -> int {
    using T = std::decay_t<decltype(val)>;
    if constexpr (std::is_same_v<T, int>) return val;
    else if constexpr (std::is_same_v<T, std::string>)
        { try { return std::stoi(val); } catch(...) { return 0; } }
    else return 0; // Muaf
}, n);
```

Görev 1.3 – Derleme ve Çalıştırma (5 dk)

a) `main.cpp`'de tüm struct başlıklarını dahil edin.

b) Bir `Öğrenci`, bir `Ders` ve bir `DersKaydi` nesnesi oluşturun (aggregat başlatma ile).

c) `DersKaydi::ortalama()` sonucunu yazdırın.

d) CMake ile derleyin ve çalıştırın.

Dikkat: variant Başlatma

`NotTuru` varsayılan olarak ilk türü (`int`) tutar. `Muaf{}` ile başlatmak için açıkça belirtin: `NotTuru vize = 85;` veya `NotTuru odev = Muaf{};`

2 Çalışma 2: Depo<K,V> ve Servis Katmanı (25 dk)

Ön Bilgi – Template Sınıf + optional/exception

`Depo<K,V>`, `std::map<K,V>` üzerine kurulmuş genel amaçlı bir depolama sınıfıdır. Anahtar-değer çiftlerini saklar, arama sonucu olarak `std::optional<V>` döndürür (bulunamazsa `nullopt`). Kesin beklentiler için `getir()` metodu istisna fırlatır.
Servis katmanı birden fazla depoyu koordine eder ve iş mantığını (ortalama hesaplama, transkript oluşturma) uygular.

Başlangıç `include/depo.h` iskeleti:

```
#pragma once
#include <map>
#include <optional>
#include <stdexcept>

template <typename K, typename V>
class Depo {
public:
    void ekle(const K& anahtar, const V& deger) {
        if (_elemanlar.contains(anahtar))
            throw std::runtime_error("Eleman zaten mevcut!");
        _elemanlar.emplace(anahtar, deger);
    }

    // GOREV 2.1: bul(), getir(), temizle() ekleyin

    std::size_t boyut() const { return _elemanlar.size(); }
    bool bos_mu() const { return _elemanlar.empty(); }
    bool iceriyor_mu(const K& a) const {
        return _elemanlar.contains(a);
    }
    const std::map<K,V>& tumunu_al() const {
        return _elemanlar;
    }
    auto begin() const { return _elemanlar.cbegin(); }
    auto end() const { return _elemanlar.cend(); }

private:
    std::map<K, V> _elemanlar;
};
```

Görev 2.1 – Depo'ya bul / getir / temizle Ekle (10 dk)

`Depo<K,V>` sınıfına aşağıdaki metotları ekleyin:

a) `std::optional<V> bul(const K& anahtar) const` – Anahtar varsa değeri `optional` içinde döndür, yoksa `std::nullopt`.

b) `const V& getir(const K& anahtar) const` – Anahtar varsa const referans döndür, yoksa `std::runtime_error` fırlat.

c) `void temizle()` – Tüm elemanları sil (`_elemanlar.clear()`).

```
// main.cpp'de test:
Depo<int, std::string> depo;
depo.ekle(1, "Ali");
auto s = depo.bul(1);        // optional<string>{"Ali"}
auto y = depo.bul(99);       // nullopt
const auto& v = depo.getir(1); // "Ali"
```

Görev 2.2 – OBYSServis ile Ders Listesi ve Sınıf Ortalaması (15 dk)

`include/servisler/obys_servis.h` ve `src/servisler/obys_servis.cpp` dosyalarını oluşturun:

a) `OBYSServis` sınıfı üç `Depo` referansı alsın (constructor'da).

b) `öğrenci_ekle()` ve `ders_ekle()` metotlarını yazın. Parametre olarak `std::string_view` kullanın.

c) `not_ekle(int öğrenci_no, string_view ders_kodu, int vize, int final_notu, int odev)` metodunu yazın.

d) `float sinif_ortalamasi(string_view ders_kodu) const` metodunu yazın. `views::values | views::filter` pipeline'ı ile ilgili dersin kayıtlarını filtreleyin ve ortalamalarını hesaplayın.

e) `main.cpp`'ye birkaç öğrenci, ders ve not ekleyip sınıf ortalamasını yazdırın.

f) CMakeLists.txt'ye `src/servisler/obys_servis.cpp`'yi ekleyin.

Ranges Pipeline ile Filtreleme

`map`'ten değerleri çekip filtrelemek için:

```
auto filtreli = kayitlar
    | std::views::values
    | std::views::filter([&kod](const DersKaydi& k) {
          return k.ders_kodu == kod;
      });
```

3 Çalışma 3: Binary Dosyalama (30 dk)

Ön Bilgi – reinterpret_cast ve Binary I/O

Binary dosyalama, verileri metin yerine ham bayt olarak saklar. `reinterpret_cast<const char*>(&veri)` ile bir değişkenin bellek adresini `char*`'a dönüştürüp `ofstream::write` ile yazarız. Okumada `ifstream::read` ile aynı baytları geri alırız.
POD türler (`int`, `float`, `char`) doğrudan `sizeof(T)` bayt yazılır. **string** için önce boyutu, sonra veriyi yazarız. **variant** için önce `index()`'i, sonra aktif türün değerini yazarız.

Başlangıç `include/dosyalama.h` iskeleti:

```cpp
#pragma once
#include <fstream>
#include <string>
#include <cstdint>

// Genel: POD turleri do rudan yaz/oku
template <typename T>
void dosyaya_yaz(std::ofstream& dosya, const T& veri) {
    dosya.write(reinterpret_cast<const char*>(&veri),
                sizeof(T));
}

template <typename T>
void dosyadan_oku(std::ifstream& dosya, T& veri) {
    dosya.read(reinterpret_cast<char*>(&veri), sizeof(T));
}

// string: once boyut, sonra veri
template <>
void dosyaya_yaz<std::string>(std::ofstream& dosya,
                              const std::string& veri) {
    std::size_t boyut = veri.size();
    dosya.write(reinterpret_cast<const char*>(&boyut),
                sizeof(boyut));
    dosya.write(veri.data(),
                static_cast<std::streamsize>(boyut));
}

template <>
void dosyadan_oku<std::string>(std::ifstream& dosya,
                               std::string& veri) {
    std::size_t boyut = 0;
    dosya.read(reinterpret_cast<char*>(&boyut),
               sizeof(boyut));
    veri.resize(boyut);
    dosya.read(veri.data(),
               static_cast<std::streamsize>(boyut));
}
```

Görev 3.1 – Öğrenci Struct Serileştirme (10 dk)

`dosyalama.h`'ye `Öğrenci` için özel `dosyaya_yaz` ve `dosyadan_oku` fonksiyonları yazın:

a) `template<> void dosyaya_yaz<Öğrenci>(...)` tanımlayın.

b) Her alanı sırayla yazın: `numara` (POD), `isim` (string), `soyisim` (string), `dogum_tarihi` (string), `cinsiyet` (string).

c) Aynı sırayla `dosyadan_oku` fonksiyonunu yazın.

d) `main.cpp`'de bir `Öğrenci` nesnesini dosyaya yazıp geri okuyun, alanları karşılaştırın.

```
// Test:
  renci    ogr{2001, "Ali", "Veli", "2000-01-01", "erkek"};
std::ofstream dos("test.bin", std::ios::binary);
dosyaya_yaz(dos, ogr);
dos.close();

  renci    okunan;
std::ifstream dis("test.bin", std::ios::binary);
dosyadan_oku(dis, okunan);
// okunan.isim == "Ali"  -> dogru!
```

Görev 3.2 – variant (NotTuru) Serileştirme (10 dk)

`dosyalama.h`'ye `NotTuru` (variant) için serileştirme ekleyin:

a) **Yazma:** Önce `index()`'i `uint8_t` olarak yazın (1 bayt), sonra `std::visit` ile aktif türün değerini yazın.

b) **Okuma:** Önce `uint8_t idx` okuyun, sonra `switch(idx)` ile doğru türü okuyup variant'a atayın.

c) Test edin: `NotTuru n1 = 85;` ve `NotTuru n2 = Muaf{};` yazıp okuyun, `index()` ve değerleri doğrulayın.

Görev 3.3 – Depo Serileştirme (10 dk)

`dosyalama.h`'ye `map<K,V>` ve `Depo<K,V>` serileştirmesi ekleyin:

a) `map<K,V>`: Önce `size()` yazın, sonra her anahtar-değer çiftini sırayla yazın.

b) `Depo<K,V>`: `tumunu_al()` ile iç map'i yazın.

c) Okumada: `Depo`'yu `temizle()` ile sıfırlayıp her çifti `ekle()` ile geri yükleyin.

d) Yardımcı fonksiyonlar:

```
template <typename K, typename V>
bool dosyaya_kaydet(const std::string& dosya_adi,
                    const Depo<K,V>& depo);
template <typename K, typename V>
bool dosyadan_yukle(const std::string& dosya_adi,
                    Depo<K,V>& depo);
```

e) `Depo<int, Öğrenci>`'ya 2 öğrenci ekleyip dosyaya kaydedin, yeni bir depoya yükleyip boyutlarını karşılaştırın.

Dikkat: Binary Dosya Açma Modları

Dosyaları her zaman `std::ios::binary` modunda açın. Aksi halde Windows'ta satır sonu dönüşümleri veriyi bozabilir:

```
std::ofstream dos("veri.bin", std::ios::binary);
```

4 Çalışma 4: Test ve Entegrasyon (25 dk)

Ön Bilgi – assert ile Birim Test ve AAA Kalıbı

`assert(kosul)` makrosu, koşul `false` ise programı durdurur ve hata mesajı gösterir. Birim testler **AAA kalıbı** ile yazılır:
Arrange (hazırla): test verilerini oluştur. **Act** (çalıştır): test edilecek işlemi yap. **Assert** (doğrula): beklenen sonucu kontrol et.
Test hedefi CMakeLists.txt'ye ayrı bir `add_executable` ile eklenir.

Görev 4.1 – Dosyalama Round-Trip Testi Yaz (10 dk)

`src/test_main.cpp` dosyası oluşturun ve aşağıdaki testleri yazın:

a) `test_dosyalama_öğrenci()`: Bir `Öğrenci` nesnesini dosyaya yazıp geri okuyun, tüm alanları `assert` ile doğrulayın.

b) `test_dosyalama_variant()`: Üç farklı `NotTuru` değeri (`int`, `string`, `Muaf`) dosyaya yazıp geri okuyun. Her birinin `index()` ve değerini doğrulayın.

c) `test_dosyalama_depo()`: `Depo<int, Öğrenci>`'ya 2 öğrenci ekleyin, `dosyaya_kaydet` ile kaydedin, `dosyadan_yukle` ile yeni depoya yükleyin, boyut ve değerleri doğrulayın.

d) Her testin sonunda geçici dosyayı `std::remove()` ile silin.

e) CMakeLists.txt'ye `TestUygulama3` hedefini ekleyin.

```
// CMakeLists.txt'ye ekleyin:
add_executable(TestUygulama3
    src/test_main.cpp
    src/servisler/obys_servis.cpp
)
```

Görev 4.2 – main.cpp'ye Dosyalama Akışı Ekle (15 dk)

`main.cpp`'yi güncelleyerek tam dosyalama entegrasyonu ekleyin:

a) Dosya isimlerini sabitler olarak tanımlayın: `const std::string DOSYA_OGRENCI = "öğrenciler.bin";`

b) Program başında `std::filesystem::exists()` ile dosyaların var olup olmadığını kontrol edin.

c) Varsa `dosyadan_yukle()` ile üç depoyu yükleyin, yoksa hardcoded veriler girin (en az 3 öğrenci, 2 ders, birkaç not).

d) Sorguları çalıştırın: ders listesi, sınıf ortalaması, transkript.

e) Program sonunda `dosyaya_kaydet()` ile üç depoyu kaydedin.

f) Programı iki kez çalıştırın: ilkinde veri oluşturulmalı, ikincisinde dosyadan yüklenmeli.

Dosya Kontrolü

`<filesystem>` başlığı ile dosya varlığını kontrol edebilirsiniz:

```
#include <filesystem>
if (std::filesystem::exists("   renciler   .bin")) {
    // dosyadan yukle
}
```

Bonus: Hata Yönetimi Entegrasyonu (10 dk)

Dosyalama katmanına hata yönetimi ekleyin:

a) `dosyadan_yukle()` başarısız olursa (dosya bozuk, eksik veri), `OBYSHatasi` istisnası fırlatın: `throw OBYSHatasi("Dosya okuma hatasi: " + dosya_adi);`

b) `main.cpp`'de `try/catch` bloğu ile yakalayın. Hata durumunda depoları `temizle()` ile sıfırlayıp hardcoded verilerle devam edin.

c) Bozuk bir dosya oluşturup (örn. birkaç rastgele bayt yazarak) hata yönetimini test edin.

Teslim Bilgileri

Teslim edilecekler:

1. `CMakeLists.txt` – Proje yapılandırması (Görev 1.1)
2. `include/varliklar/ders_kaydi.h` – DersKaydi struct'ı (Görev 1.2)
3. `include/depo.h` – Depo template sınıfı (Görev 2.1)
4. Servis dosyaları (Görev 2.2)
5. `include/dosyalama.h` – Binary serileştirme (Görev 3.1–3.3)
6. `src/test_main.cpp` – Birim testleri (Görev 4.1)
7. `src/main.cpp` – Ana program (Görev 4.2)
8. *Bonus:* Hata yönetimi entegrasyonu

Puanlama: Çalışma 1: 20p Çalışma 2: 25p Çalışma 3: 30p Çalışma 4: 25p Bonus: 10p